

尋形 Conform

框選一個形，找出所有同形——先進封裝 DXF 的幾何比對與設計規則檢查

Frame-select one pattern, find every instance — geometry matching & DRC for advanced packaging

480 → 1

等效人工工時 → 機器 1 小時

100%

選定圖樣全 entity 覆蓋 (vs 抽樣)

820,212

單張真實 DXF 實體數，秒級互動

驗證瓶頸：先進封裝設計檢查的痛點

The Verification Bottleneck — why Conform exists

先進封裝後段驗證：一個漏檢的設計錯誤，可能造成基板重投、封裝組裝良率損失，甚至流到客戶端釀成客退——良率與可靠度的最後一道防線。

人力與時間

6 位資深工程師
× 超過 10 天

最貴的人力，綁在最低槓桿的重複量測上

覆蓋率

抽樣式驗證
缺陷會漏

抽樣看不完 → 未覆蓋區就是漏檢風險

授權瓶頸

僅 4 套 AutoCAD
併發授權

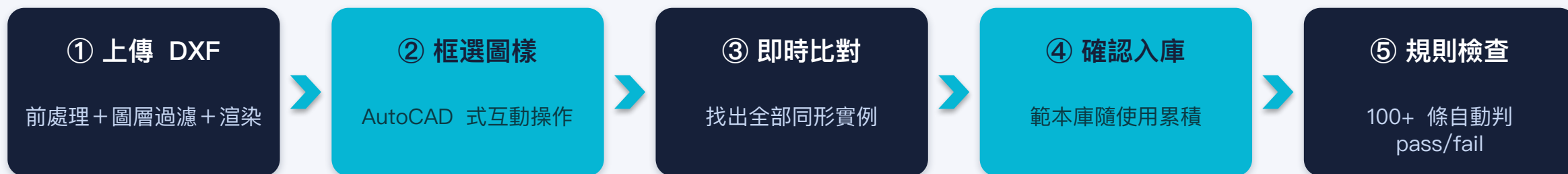
授權數 = 驗證並行度上限，全隊迭代卡死

這三個瓶頸是結構性的，不是效率問題——加人、加班、買授權只能緩解、無法消除，得換驗證方式。

解法：框選即查詢——從框選到規則報告的閉環

Frame-select → match → rule check, one closed loop

工程師在瀏覽器裡框選一個圖樣，系統即時找出全部同形、確認後入庫；範本庫隨使用累積，之後新圖一鍵全掃、直出報告。



零學習成本 FAMILIAR UX

互動完全比照 AutoCAD：中鍵拖曳平移、左→右 window / 右→左 crossing 框選。

驗證工程師不需重新學一套工具，開瀏覽器就能上手。

FLYWHEEL

範本庫飛輪：越用越省

每次確認的圖樣都累積進範本庫（現有 50+ 範本）；

新圖上傳後一鍵 scan-all 全範本掃描——第一張圖教過的，之後每張圖都自動會。

閉環的關鍵在第③步「即時比對」——在 82 萬個實體裡找出全部同形，還要秒級回應。下一頁講怎麼辦到。

像找星系一樣，在圖海中找出每一個同形

The Matching Engine & its astronomy inspiration

閉環的第③步：在整張 DXF 裡找出框選圖樣的**全部同形實例**。難點只有一個：**規模**。

暴力解的代價 BRUTE FORCE

遍歷：每個 entity 都要兩兩比較 $\rightarrow O(N^2)$

但 pattern 最多由 $k = 120$ 個 entity 組成，每個都要比 $\rightarrow$ 再 $\times k$
 $\rightarrow O(N^2 \cdot k)$ 量級 $\approx 8.1 \times 10^{13}$ 次 / 模板

真實 DXF $N \approx 820,212$ ；每次還要旋轉對位 + 容差比對——遍歷根本跑不完。

INSPIRATION

靈感：天文學家怎麼找星系 / 星團 / 星雲？

不逐顆星互比（那是宇宙級 $O(N^2)$ ）——而是：

- ① 旋轉 / 縮放 / 平移都不變的幾何簽章
- ② 空間索引快速定位候選天區
- ③ 只對倖存候選做精確比對

（同類思路：astrometry.net quad-hash、FoF 分群）

把同一招搬到 DXF：同形圖樣 = 一個星座，整張圖 = 一片星空

① 幾何簽章 gate

旋轉 / 鏡像 / 縮放不變

② cKDTree 空間索引

$O(\log N)$ 鄰域

③ PCA 主軸對位

sign-variant 擇優

④ 對稱 chamfer

容差評分

⑤ inverted-index

內含抑制 $\rightarrow$ 近線性

遍歷 brute-force $O(N^2 \cdot k) \approx 8.1 \times 10^{13}$ 次 / 模板 ($k \leq 120$) $\rightarrow$ 單核估算約 數週 ~ 數月

尋形 Conform 簽章 gate + cKDTree 空間索引剪枝到近線性 $\rightarrow$ 互動式秒級回應

成效： 6 人 10 天 → 1 人 1 小時

Results & Impact — the core metric

VERIFY ONE DRAWING

6 人 × 10 天 → **1 人 × 1 小時**
= 約 480 等效人工工時 (6 人 × 10 天 × 8h) → 約 1 小時機器運行

≈ 480 → 1
人力投入約 1/480 (工程師小時)

維度	BEFORE (舊流程)	AFTER (尋形 Conform)
人力 / 工時	6 位資深工程師 × 10 天	1 位工程師 × 1 小時 (含操作 + 100+ 條自動化條件檢查)
覆蓋率	抽樣 (缺陷會漏)	100% 全 entity 全覆蓋 + 100+ 條自動化檢查 (架構保證)
併發 / 吞吐	4 套 AutoCAD 授權序列化	多圖同時處理, 不被單張圖卡住; 零 AutoCAD 授權

「抽樣 → 選定圖樣 100% 全掃」是品質階躍——漏檢從機率, 變成可量測的覆蓋率。

生產級規模 | 真實 DXF 820,212 實體
內含抑制 20s → 8ms (實測)

LLM 時代，怎麼做出「對」的產品？

Anyone can ship with an LLM — shipping the right thing is the hard part

LLM 把「寫程式」變便宜了，卻沒有把「做對產品」變便宜——需求、品質、AI 的發揮，仍然取決於工程紀律與工具。

Q1 真的符合需求嗎？

規格先行

意圖先寫成 proposal / design，在紙上吵完再動手——

方向錯了在規格層修，成本是幾行字；
等 diff 出來才修，成本是整個功能重寫。

Q2 品質怎麼確保？

流程強制

品質不是宣稱，是閘門——機器驗證的能力契約、

605 個測試、真瀏覽器端到端驗證，
每一關不過就不能 merge。

Q3 怎麼讓 AI 表現更好？

給對工具

codegraph 給它導航、playwright 給它眼睛、

skills 把流程固化成生產線——
AI 負責提案，工程判斷負責把關。

這三個問題，就是接下來兩頁——尋形 Conform 的開發生產線與品質工程，全部圍繞它們設計。

用 Claude Code 把開發變成生產線

Building Conform as a Claude Code Production Line



Skills | 自建技能生態 規格流 spectra-* x12 · 測試 tdd · 審查 grill-me / diagnose / triage
規劃 to-prd / to-issues · 領域 add-rule · 雛型 prototype · 架構 improve-codebase-architecture 等

AI 提案，工程判斷把關——每個採用都附一個被我否決的替代方案

- 採用** 我否決 Claude 的「密度啟發式」，改採確定性互斥視圖規則——可重現、operator 看得懂
- 後來推翻** 密度仲裁上線後產線 17,482 球整批誤判，我判定不追不可重現的 bug、整個子系統當死碼砍掉
- 被產線打臉再修** design 判 $O(M^2)$ 「便宜」被 20k 球產線打臉到 20s+，我換 inverted index 近線性（6000 球 2.0s→8ms）
- 延後** Postgres / MinIO 遷移我寫好 ADR 但標「未實作」——_jobs 才是硬牆，無 HA 需求前不動

55 archived + 34 active changes · 10 capability specs · 290 commits · 兩個月（2026-05-13 起）

程式品質 × 規格驅動工程

Code Quality × Spec-Driven Development

規格驅動：品質由流程強制，非口頭宣稱

- ▶ 10 個機器驗證能力契約（SHALL + WHEN/THEN 場景）
- ▶ 89 個 OpenSpec changes（55 archived + 34 active）
- ▶ openspec validate --specs 為 pre-merge 閘門
- ▶ 每個 change 一分支、留書面決策與審計軌跡

工程硬證據（每條釘 file / test）

- ✓ 605 測試函式、測試 13.3k LOC > 應用 12.9k LOC
- ✓ 倒排索引：含括抑制 $O(N^2)$ → 近線性（守 $N=20000 < 2s$ ）
- ✓ AST 不變量測試：workers 永不讀 stale cache
- ✓ DRC envelope 嚴格驗證：壞輸出 → 大聲 400，不靜默
- ✓ canvas.js ↔ Python 常數 drift-guard 測試

MATCHER PCA 主軸對位 + sign-variant 擇優 + 對稱 chamfer → 平移 / 旋轉 / 鏡像 / $\pm 5\%$ 縮放不變

三條「真實 bug 驅動」fast path（820,212 實體 DXF bucket-split、 $O(N^2)$ 抑制爆 20s）—— bug 驅動的演算法，不是紙上設計。

規格驅動讓「不同意」變便宜——我們在 design.md 的 ## Decisions 吵完每個 Alternative，而不是等 diff 出來才回頭。

創新差異化與未來擴展

Differentiation & Roadmap

vs 舊做法 人工逐物件抽樣量測（AutoCAD 內手動） → 框選 → 比對 → 規則檢查的閉環（class-agnostic 幾何引擎，非封裝寫死）



先進封裝 DXF

已實作

PCB Gerber / ODB++ 間距 DRC

可擴展

GDSII / OASIS layout 圖樣列舉

可擴展

photomask / MEMS 圖樣稽核

可擴展

interposer / RDL 軟板佈局檢查

可擴展

生產化進行中 MariaDB 多副本 · MinIO 物件儲存 · Keycloak SSO 三級權限 · 產品版本化 · 簽核存證（openspec 34 active changes）
通用定位 DRC 引擎為 adapter + 版本化 bundle → 可獨立出貨；「幾何→規則引擎」通用前端層，可餵任意 DRC 後端。



Q & A

6 人 × 10 天 → 1 人 × 1 小時 · 選定圖樣 100% 覆蓋 · 零 AutoCAD 授權依賴

演算法細節

簽章 gate / cKDTTree / PCA 對位 → P4

數字怎麼算

480 工時換算與量級估算 → P5 備註

AI 怎麼用

採用 / 否決決策日誌 → 下一頁 BACKUP

採用 / 否決 決策日誌

AI-proposed / human-decided log (Q&A backup)

改進主題	Claude 建議	我的決定	我們怎麼討論 / 理由	結果
同半徑 BGA 球 vs 對位點	鄰域密度仲裁 (自動推 pitch、鄰居數分類)	後來推翻	初期採用 (無參數)；產線 17,482 球大陣列整批誤判成 Fiducial、合成案重現不出 → 不追不可重現的啟發式 bug	改用視圖約束
替代分類規則	互斥視圖判定 (BGA= 底視圖、Fiducial= 頂視圖)	採用	確定性、operator 看得懂、免疫密度 edge case；明確否決「保留密度群當保險」	上線穩定 (77e8832)
失效仲裁子系統去留	保留 arbitrate() 當未來骨架	改寫 (YAGNI 刪)	視圖約束讓 registry 清空、call site 全成 pass-through → 可達但無作用的死碼，直接刪	子系統移除 (fffd30e)
含括重複比對抑制效能	每類 pairwise 掃描, design 判 $O(M^2)$ 「便宜」	採用後修正	上線打臉：20k 球單這步 20s+ → 換 inverted handle-index (X 的超集必含 X 全部 handle)，近線性	6000 球 2.0s→8ms (13f179f)
多 DXF 一角色的 view 維度	引入 dxf_view enum，正交於設計角色 (Decision 1-7)	後來推翻	Decision 8 簡化轉向：使用者回饋主流程是一檔含 top/bottom/side region，分割軸未必對齊 → enum 是 over-fit	全檔 multi，欄位 vestigial
基板比對策略	新增 per-class match_strategy (chamfer/signature)	採用 (取代被否決路徑)	前案 per-class tolerance 已 revert (9db06a6→c4df21d)：tolerance 橋不過固定 ±20% pre-filter	match_strategy 正式取代
上傳上限 SEC-001 預檢	加請求層 Content-Length 預檢, buffer 前拒絕	延後	multipart 的 Content-Length 涵蓋整個 body 非單檔, FastAPI 無 per-part length；對內部信任使用者邊緣價值低	緩衝後 len() 為權威防線

主軸：「AI 提案，工程判斷把關——每個採用都附一個被我否決的替代方案」。錨點均經 git/reflog 核對；效能絕對數字以 design.md / 專案備忘為準，非現場 benchmark。